

PROJECT REPORT

Subject: Deployment of Django REST API on Azure with Hybrid Infrastructure

1. Purpose and Scope

This report documents the process of taking the Django REST API project, which I named PostgreDjango, from the development environment (local machine) and migrating it to a fully automated (CI/CD) and scalable structure on the Azure cloud platform. The project's architecture was designed using a hybrid model that combines PaaS (Platform as a Service) for the application layer and IaaS (Infrastructure as a Service) for the database layer.

2. Phase: Infrastructure Preparation and Database Setup (IaaS)

As a first step, I created a secure virtual machine (VM) on Azure to host our API's PostgreSQL database.

Virtual Machine (VM) Creation: I provisioned a virtual machine (Ubuntu 24.04) named onurMachine through the Azure portal.

Firewall (NSG) Configuration (SSH): To manage the VM, I allowed SSH (Port 22) connections from my own IP address in the Network Security Group (NSG) rules.

PostgreSQL Installation: I connected to the server via SSH and installed the PostgreSQL 16 server using the apt package manager.

Database and User Creation: To ensure my application connects securely, I used the psql interface to create a database named djangoapp and a user named admin.

onurMachine | Network settings

Virtual machine

Search

Connect

Networking

Network settings

Load balancing

Application security groups

Network manager

Settings

Disks

Extensions + applications

Operating system

Configuration

Advisor recommendations

Properties

Locks

Availability + scale

Security

Backup + disaster recovery

Operations

Monitoring

Automation

Help

onurmachine157_z1 (primary) / ipconfig1 (primary)

Essentials

Network interface : onurmachine157_z1

Virtual network / subnet : vnet-west europe / snet-west europe-1

Public IP address : 50.85.248.188

Private IP address : 172.16.0.4

Admin security rules : 0 (Configure)

Load balancers : 0 (Configure)

Application security groups : 0 (Configure)

Network security group : onurMachine-nsg

Accelerated networking : Disabled

Effective security rules : 0

Rules

Network security group onurMachine-nsg (attached to networkinterface: onurmachine157_z1)

Impacts 0 subnets, 1 network interfaces

Search rules

Source == all

Destination == all

Protocol == all

Action == all

Port == all

Priority	Name	Port	Protocol	Source	Destination	Action
Inbound port rules (5)						
102	SSH	22	TCP	31.223.99.108	Any	Allow
112	Allow PostgreSQL Inbound	5432	TCP	9.163.251.66,132.220.69.132,50.85.6.2...	Any	Allow
65000	AllowVnetInBound	Any	Any	VirtualNetwork	VirtualNetwork	Allow
65001	AllowAzureLoadBalancerInBound	Any	Any	AzureLoadBalancer	Any	Allow
65500	DenyAllInBound	Any	Any	Any	Any	Deny
Outbound port rules (3)						

Step 3: Database Accessibility and Testing

After installation, I made this database accessible both from my local machine and from the App Service that will be installed later.

PostgreSQL Configuration: I edited the postgresql.conf file so that connections are accepted not only from localhost but also from outside (listen_addresses = '*').

Access Control: I configured the pg_hba.conf file so that the admin user can connect to the djangoapp database from any IP address with a password (md5).

NSG Configuration (PostgreSQL): I added a new rule to the VM's security group (NSG) allowing requests from my local IP address to the PostgreSQL port 5432.

Local Connection Test: I ran the py manage.py migrate command by pointing my settings.py file to the VM's Public IP. After some debugging (case sensitivity and pg_hba.conf settings), I successfully connected to the database on the Azure VM from my local machine and created the tables.

```
no vm guests are running outdated hypervisor (qemu) binaries on this host.
azureuser@onurMachine:~$ sudo -i -u postgres
postgres@onurMachine:~$ psql
psql (16.10 (Ubuntu 16.10-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# CREATE USER admin WITH PASSWORD 'admin';
CREATE ROLE
postgres=# CREATE DATABASE djangoApp OWNER admin;
CREATE DATABASE
postgres=# q
postgres=# \q
postgres@onurMachine:~$ exit
logout
azureuser@onurMachine:~$ sudo nano /etc/postgresql/16/main/postgresql.conf
[sudo] password for azureuser:
azureuser@onurMachine:~$ sudo nano /etc/postgresql/16/main/postgresql.conf
azureuser@onurMachine:~$ sudo nano /etc/postgresql/16/main/pg_hba.conf
azureuser@onurMachine:~$ sudo nano /etc/postgresql/16/main/pg_hba.conf
azureuser@onurMachine:~$ sudo systemctl restart postgresql
azureuser@onurMachine:~$ exit
logout
Connection to 50.85.248.188 closed.
Last login: Fri Nov 7 12:50:43 2025 from 51.225.99.100
azureuser@onurMachine:~$ sudo -i -u postgres
[sudo] password for azureuser:
[sudo] password for azureuser:
Sorry, try again.
[sudo] password for azureuser:
postgres@onurMachine:~$ psql
psql (16.10 (Ubuntu 16.10-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# \l
                                List of databases
  Name      | Owner   | Encoding | Locale Provider | Collate | Ctype    | ICU Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
djangoapp   | admin   | UTF8     | libc             | C.UTF-8 | C.UTF-8  |             |           |
postgres    | postgres | UTF8     | libc             | C.UTF-8 | C.UTF-8  |             |           |
template0   | postgres | UTF8     | libc             | C.UTF-8 | C.UTF-8  |             |           |
template1   | postgres | UTF8     | libc             | C.UTF-8 | C.UTF-8  |             |           |
(4 rows)
```

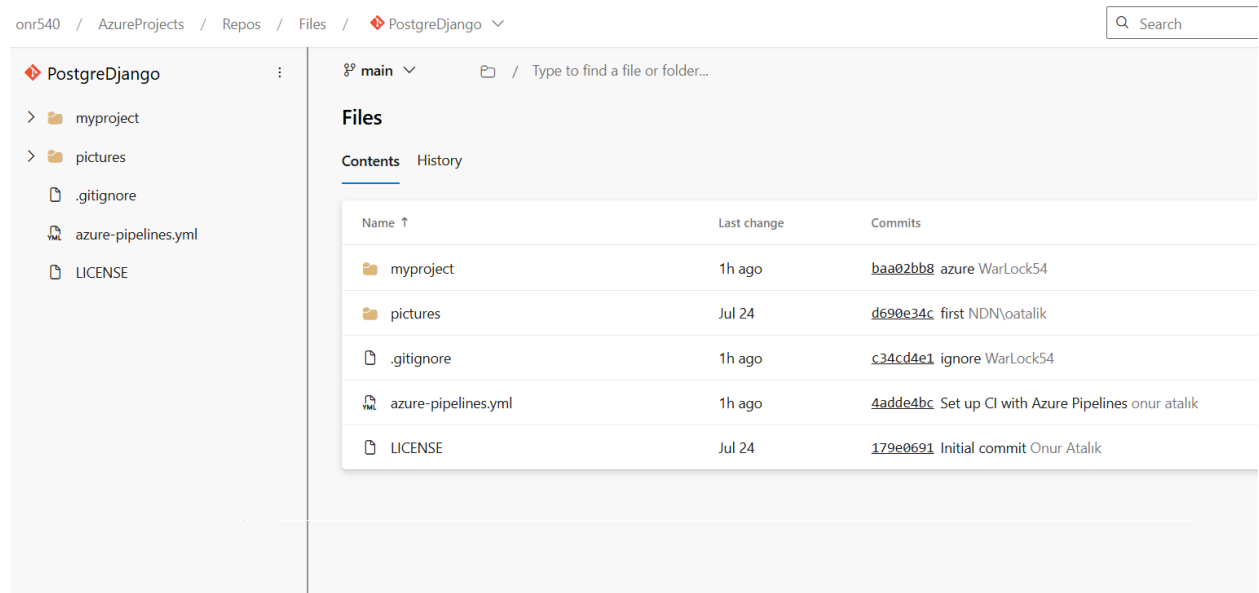
Name	Owner	Encoding	Locale Provider	Collate	Ctype	ICU Locale	ICU Rules	Access privileges
djangoapp	admin	UTF8	libc	C.UTF-8	C.UTF-8			
postgres	postgres	UTF8	libc	C.UTF-8	C.UTF-8			
template0	postgres	UTF8	libc	C.UTF-8	C.UTF-8			=c/postgres +
template1	postgres	UTF8	libc	C.UTF-8	C.UTF-8			=c/postgres +

```
postgres=# \l
                                List of databases
  Name      | Owner   | Encoding | Locale Provider | Collate | Ctype    | ICU Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
djangoapp   | admin   | UTF8     | libc             | C.UTF-8 | C.UTF-8  |             |           |
postgres    | postgres | UTF8     | libc             | C.UTF-8 | C.UTF-8  |             |           |
template0   | postgres | UTF8     | libc             | C.UTF-8 | C.UTF-8  |             |           |
template1   | postgres | UTF8     | libc             | C.UTF-8 | C.UTF-8  |             |           |
(4 rows)
```

Step 4: Preparing the Project for Production

After verifying the database connection, I prepared my project for automatic deployment (CI/CD) via Azure DevOps.

Security (settings.py): I updated my settings.py file to read all sensitive information (e.g., SECRET_KEY, DATABASE_URL) from environment variables using os.environ.get and the dj_database_url library. This prevented sensitive information from being pushed to the code repository.



Step 4: Preparing the Project for the Production Environment

After verifying the database connection, I prepared my project for automatic deployment (CI/CD) via Azure DevOps.

Security (settings.py): I updated my settings.py file to read all secrets (e.g., SECRET_KEY, DATABASE_URL) from environment variables using os.environ.get and the dj_database_url library. This prevented secrets from being committed to the code repository.

Startup Script (startup.sh): I wrote a startup.sh script that defines how App Service should start the application. This script automatically runs the migrate command and starts gunicorn every time the server starts.

Code Repository (Azure Repos): I pushed the project to the new Azure DevOps repository (PostgreDjango) I created using git.

Step 5: Deployment and Automation (CI/CD)

As the final step, I configured the web server where my API will run 24/7 and the pipeline that will automatically push my code to this server.

App Service (PaaS) Setup: I created an Azure App Service (Linux, Python 3.11) named onurDjango for my API to run.

Environment Variables: In the Environment variables section of the App Service, I entered DATABASE_URL and other secret settings such as DJANGO_SECRET_KEY so that it could connect to my database on the VM.

Startup Command: In the App Service's Configuration -> General settings section, I entered the bash startup.sh command to run my startup.sh file.

CI/CD Pipeline Setup: In Azure DevOps, I created the azure-pipelines.yml file using the “Python to Linux Web App” template. This pipeline is configured to automatically deploy the code to the onurDjango App Service whenever a push is made to the main branch.

Final NSG Setting: I copied the “Outbound IPs” from the Networking menu of the onurDjango App Service and granted access to port 5432 on the onurMachine (VM) for these IPs.

Step 6. Results and Current Status

All technical configuration steps have been successfully completed. The network connection between the application (App Service) and the database (VM) has been established, settings.py has been secured, and the CI/CD pipeline is ready for deployment.